

数值算法 Homework 03

姓名: 雍崔扬

学号: 21307140051

Due: 08:00 Mar. 10, 2025

Problem 1

Suppose that a, b, c, d satisfy:

$$\begin{aligned}a + b + c + d &= 5 \\2a + 4b + 8c + 16d &= 7 \\3a + 9b + 27c + 81d &= 11 \\4a + 16b + 64c + 256d &= 1\end{aligned}$$

Calculate $5a + 25b + 125c + 625d$ by hand.

Note: Try to simplify the calculation as much as you can.

Preparation:

- **Newton 插值公式:**

$$\begin{aligned}P(x) &= f(x_0) + \sum_{k=1}^n f[x_0, x_1, \dots, x_k] \prod_{i=0}^{k-1} (x - x_i) \\&= f(x_0) + f[x_0, x_1](x - x_0) + \dots + f[x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \dots (x - x_{n-1}) \\&\text{where } f[x_0, x_1, \dots, x_k] := \sum_{i=0}^k y_i \prod_{j \neq i} \frac{1}{x_i - x_j}\end{aligned}$$

其中 $f(\cdot)$ 在 $x_0, \dots, x_k$ 上的差商 $f[x_0, x_1, \dots, x_k]$ 满足:

$$f[x_0, \dots, x_k] = \frac{f[x_1, \dots, x_k] - f[x_0, \dots, x_{k-1}]}{x_k - x_0}$$

Solution:

定义 $f(x) = ax + bx^2 + cx^3 + dx^4$, 则我们有:

(注意别把 $(0, 0)$ 遗漏了)

$$\begin{aligned}f(0) &= 0 \\f(1) &= a + b + c + d = 5 \\f(2) &= 2a + 4b + 8c + 16d = 7 \\f(3) &= 3a + 9b + 27c + 81d = 11 \\f(4) &= 4a + 16b + 64c + 256d = 1\end{aligned}$$

即我们有 $(0, 0), (1, 5), (2, 7), (3, 11), (4, 1)$ 五个数据点.

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 4 \\ f(0) & f(1) & f(2) & f(3) & f(4) \\ & f[0, 1] & f[1, 2] & f[2, 3] & f[3, 4] \\ & & f[0, 1, 2] & f[1, 2, 3] & f[2, 3, 4] \\ & & & f[0, 1, 2, 3] & f[1, 2, 3, 4] \\ & & & & f[0, 1, 2, 3, 4] \end{bmatrix} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 \\ 0 & 5 & 7 & 11 & 1 \\ & 5 & 2 & 4 & -10 \\ & & -\frac{3}{2} & 1 & -7 \\ & & & \frac{5}{6} & -\frac{8}{3} \\ & & & & -\frac{7}{8} \end{bmatrix}$$

因此根据 Newton 插值公式可知, $f(x)$ 的表达式为:

$$\begin{aligned}
f(x) &= f(0) + f[0, 1]x + f[0, 1, 2]x(x-1) + f[0, 1, 2, 3]x(x-1)(x-2) + f[0, 1, 2, 3, 4]x(x-1)(x-2)(x-3) \\
&= 0 + 5x - \frac{3}{2}x(x-1) + \frac{5}{6}x(x-1)(x-2) - \frac{7}{8}x(x-1)(x-2)(x-3) \\
&= x \cdot \left[5 - \frac{3}{2}(x-1) + \frac{5}{6}(x-1)(x-2) - \frac{7}{8}(x-1)(x-2)(x-3) \right]
\end{aligned}$$

于是我们有:

$$\begin{aligned}
5a + 25b + 125c + 625d &= f(5) \\
&= 5 \cdot \left[5 - \frac{3}{2}(5-1) + \frac{5}{6}(5-1)(5-2) - \frac{7}{8}(5-1)(5-2)(5-3) \right] \\
&= 5 + 8 + 12 - \frac{8}{3} \cdot 24 \\
&= -60
\end{aligned}$$

Problem 2

Let $f : \mathbb{R} \mapsto \mathbb{R}$ be a sufficiently smooth function.

Show that:

$$\lim_{(x_0, \dots, x_n) \rightarrow (x_*, \dots, x_*)} f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(x_*)}{n!} \quad (\forall x_* \in \mathbb{R})$$

Proof:

记 $y_i = f(x_i)$ ($i = 0, 1, \dots, n$)

记 $f(x)$ 在 $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ 上的 n 次插值多项式为:

$$P(x) := f(x_0) + f[x_0, x_1](x - x_0) + \dots + f[x_0, x_1, \dots, x_n] \prod_{i=0}^{n-1} (x - x_i)$$

$$\text{where } f[x_0, x_1, \dots, x_k] := \sum_{i=0}^k y_i \prod_{j \neq i} \frac{1}{x_i - x_j}$$

其 n 阶导数为:

$$P^{(n)}(x) = n! f[x_0, x_1, \dots, x_n]$$

注意到 $f(x) - P(x)$ 在 $\mathbb{R}$ 上至少有 $n+1$ 个零点 $x_0, x_1, \dots, x_n$

我们记:

$$a := \min\{x_0, x_1, \dots, x_n\}$$

$$b := \max\{x_0, x_1, \dots, x_n\}$$

则 $f^{(n)}(x) - P^{(n)}(x)$ 在 (a, b) 区间内至少存在一个零点, 记为 $\xi \in (a, b)$

于是我们有:

$$\begin{aligned}
f^{(n)}(\xi) - P^{(n)}(\xi) &= f^{(n)}(\xi) - n! f[x_0, x_1, \dots, x_n] = 0 \\
&\Updownarrow \\
f[x_0, x_1, \dots, x_n] &= \frac{f^{(n)}(\xi)}{n!}
\end{aligned}$$

由于 $f(x)$ 在 $\mathbb{R}$ 上是充分光滑的, 故 $f^{(n)}(x)$ 在 $\mathbb{R}$ 上是连续函数.

于是我们有:

$$\begin{aligned}
\lim_{(x_0, x_1, \dots, x_n) \rightarrow (x_*, x_*, \dots, x_*)} f[x_0, x_1, \dots, x_n] &= \lim_{a, b \rightarrow x_*} \frac{f^{(n)}(\xi)}{n!} \\
&= \lim_{\xi \rightarrow x_*} \frac{f^{(n)}(\xi)}{n!} \\
&= \frac{f^{(n)}(x_*)}{n!}
\end{aligned}$$

Problem 3

Let $p(x)$ be a polynomial of degree n with roots $\mu_1, \dots, \mu_n$.

Suppose that $\gcd(p(x), p'(x)) = 1$, i.e., $p(x)$ has no repeated roots.

Show that:

$$\sum_{i=1}^n \frac{1}{p'(\mu_i)} = 0$$

• **Lagrange 插值多项式:**

给定 $n+1$ 个点 $(x_0, y_0), \dots, (x_n, y_n)$, 其 n 阶插值多项式为:

$$p(x) = \sum_{k=0}^n y_k \prod_{i \neq k} \frac{x - x_i}{x_k - x_i}$$

Proof:

根据 $\gcd(p(x), p'(x)) = 1$ 可知 $\mu_1, \dots, \mu_n$ 互不相同且 $p(x) := \prod_{i=1}^n (x - \mu_i)$

则我们有:

$$\begin{aligned}
p'(x) &= \sum_{j=1}^n \prod_{i \neq j} (x - \mu_i) \\
&= \prod_{i \neq k} (x - \mu_i) + (x - \mu_k) \sum_{j \neq k} \prod_{i \neq j, k} (x - \mu_i) \quad (\forall k = 1, \dots, n)
\end{aligned}$$

于是我们有:

$$\begin{aligned}
p'(\mu_k) &= \prod_{i \neq k} (\mu_k - \mu_i) + (\mu_k - \mu_k) \sum_{j \neq k} \prod_{i \neq j, k} (\mu_k - \mu_i) \\
&= \prod_{i \neq k} (\mu_k - \mu_i) \\
&\neq 0
\end{aligned}$$

于是我们可以定义一个至多 $n-1$ 次的辅助多项式:

$$q(x) := \sum_{j=1}^n \frac{1}{p'(\mu_j)} \prod_{i \neq j} (x - \mu_i)$$

它满足:

$$\begin{aligned}
q(\mu_k) &= \sum_{j=1}^n \frac{1}{p'(\mu_j)} \prod_{i \neq j} (\mu_k - \mu_i) \\
&= \frac{1}{p'(\mu_k)} \prod_{i \neq k} (\mu_k - \mu_i) \quad (\forall k = 1, \dots, n) \\
&= \frac{1}{\prod_{i \neq k} (\mu_k - \mu_i)} \prod_{i \neq k} (\mu_k - \mu_i) \\
&= 1
\end{aligned}$$

由于 $q(x)$ 的次数至多为 $n - 1$, 且在 $\mu_1, \dots, \mu_n$ 这 n 个不同的点上取值为 1, 故 $q(x) \equiv 1$ 因此 $q(x)$ 的 $n - 1$ 阶项的系数为零, 即有:

$$\sum_{j=1}^n \frac{1}{p'(\mu_j)} = 0$$

Problem 4

Polynomial interpolation provides one way to approximate a given function.

For instance, let $0 = x_1 < \dots < x_n = 2\pi$ be equally spaced interpolation nodes.

The interpolation polynomial passing through all $(x_i, \sin x_i)$'s

can be used to approximate the sine function $f(x) = \sin x$.

Try to visualize the difference between the interpolation polynomial

and the sine function for a few different choices of n .

- (optional: 多项式插值的 Runge 现象)

What happens if the same technique is applied

to approximate the rational function $f(x) = (1 + 25x^2)^{-1}$ over $[-1, 1]$

(using equally spaced interpolation nodes)

(1) Newton 插值

Newton 插值公式:

$$\begin{aligned} P(x) &= f(x_1) + \sum_{k=2}^n f[x_1, x_2, \dots, x_k] \prod_{i=1}^{k-1} (x - x_i) \\ &= f(x_1) + f[x_1, x_2](x - x_1) + \dots + f[x_1, x_2, \dots, x_n](x - x_1)(x - x_2) \cdots (x - x_{n-1}) \\ \text{where } f[x_1, x_2, \dots, x_k] &:= \sum_{i=1}^k y_i \prod_{j \neq i} \frac{1}{x_i - x_j} \end{aligned}$$

实际计算时, 差商不是直接通过 $f[x_1, x_2, \dots, x_k] := \sum_{i=1}^k y_i \prod_{j \neq i} \frac{1}{x_i - x_j}$ 计算的 (否则就是 Lagrange 插值了)

它是从 $f[x_k] = f(x_k) = y_k$ 出发递归地求解 $f[x_1, \dots, x_k]$:

$$f[x_1, \dots, x_k] = \frac{f[x_2, \dots, x_k] - f[x_1, \dots, x_{k-1}]}{x_k - x_1}$$

因为这里 $x_1, x_2, \dots, x_n$ 是等距节点 (记间距为 h), 所以差商的递归公式还可简化为:

$$f[x_1, \dots, x_k] = \frac{f[x_2, \dots, x_k] - f[x_1, \dots, x_{k-1}]}{(k-1)h}$$

计算差商表的函数:

```
% 计算差商表的函数
function delta_f = divided_differences(y, h)
    % 获取数据点的数量
    n = length(y);
    % 初始化差商表
    delta_f = zeros(n, n);
    % 第一列为原始数据
    delta_f(:, 1) = y(:);

    % 计算差商表
    for j = 2:n
```

```

        for i = 1:n-j+1
            % 递归计算差商
            delta_f(i, j) = (delta_f(i+1, j-1) - delta_f(i, j-1)) / ((j-1)*h);
        end
    end

    % 返回差商表的第一行
    delta_f = delta_f(1,:);
end

```

Newton 插值函数:

```

function newton_interpolation(f, n, interval)
    x_exact = linspace(interval(1), interval(2), 1000);
    x_nodes = linspace(interval(1), interval(2), n);
    y_exact = f(x_exact); % 精确值
    y_nodes = f(x_nodes); % 等距采样
    y_interp = zeros(size(x_exact)); % 等距采样的 Newton 插值结果

    % 计算节点间距 h
    h = x_nodes(2) - x_nodes(1);
    % 计算差商表 (的第一行)
    delta_f = divided_differences(y_nodes, h);

    % 迭代项
    ones_ = ones(size(x_exact));
    term = ones_;

    % Newton 插值公式
    for j = 1:n
        y_interp = y_interp + delta_f(j) * term ;
        term = term .* (x_exact - x_nodes(j));
    end

    % 插值与精确值的差距
    y_diff = abs(y_interp - y_exact);

    % 绘图
    % 左侧纵轴: 绘制精确值和插值结果
    yyaxis left;
    plot(x_exact, y_exact, 'k-', 'Linewidth', 1.5); hold on;
    plot(x_exact, y_interp, 'r--', 'Linewidth', 1);
    scatter(x_nodes, y_nodes, 25, 'k', 'filled');
    title(['Newton Interpolation with n = ', num2str(n)]);
    hold off;

    % 右侧纵轴: 绘制 log(y_diff)
    yyaxis right;
    plot(x_exact, log(y_diff), 'b-.', 'Linewidth', 1);
    ylabel('log(y_{diff})');
    legend('Exact', 'Interpolated', 'Nodes', 'log difference');
    grid on;
end

```

函数调用:

```
f = @(x) sin(x);
```

```

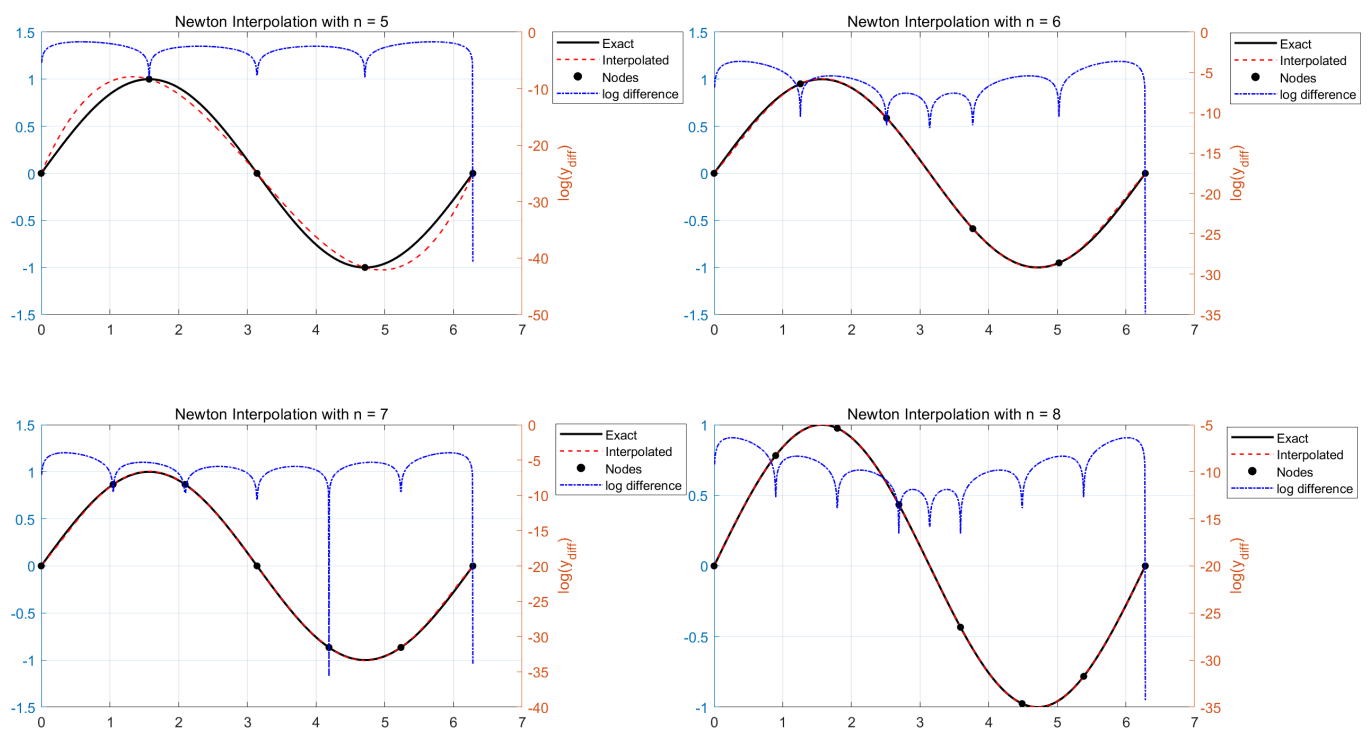
g = @(x) 1 ./ (1 + 25 * x.^2);
ns_f = [5, 6, 7, 8];
ns_g = [5, 10, 15, 20];

figure;
for i = 1:length(ns_f)
    subplot(2,2,i);
    newton_interpolation(f, ns_f(i), [0, 2*pi]);
end

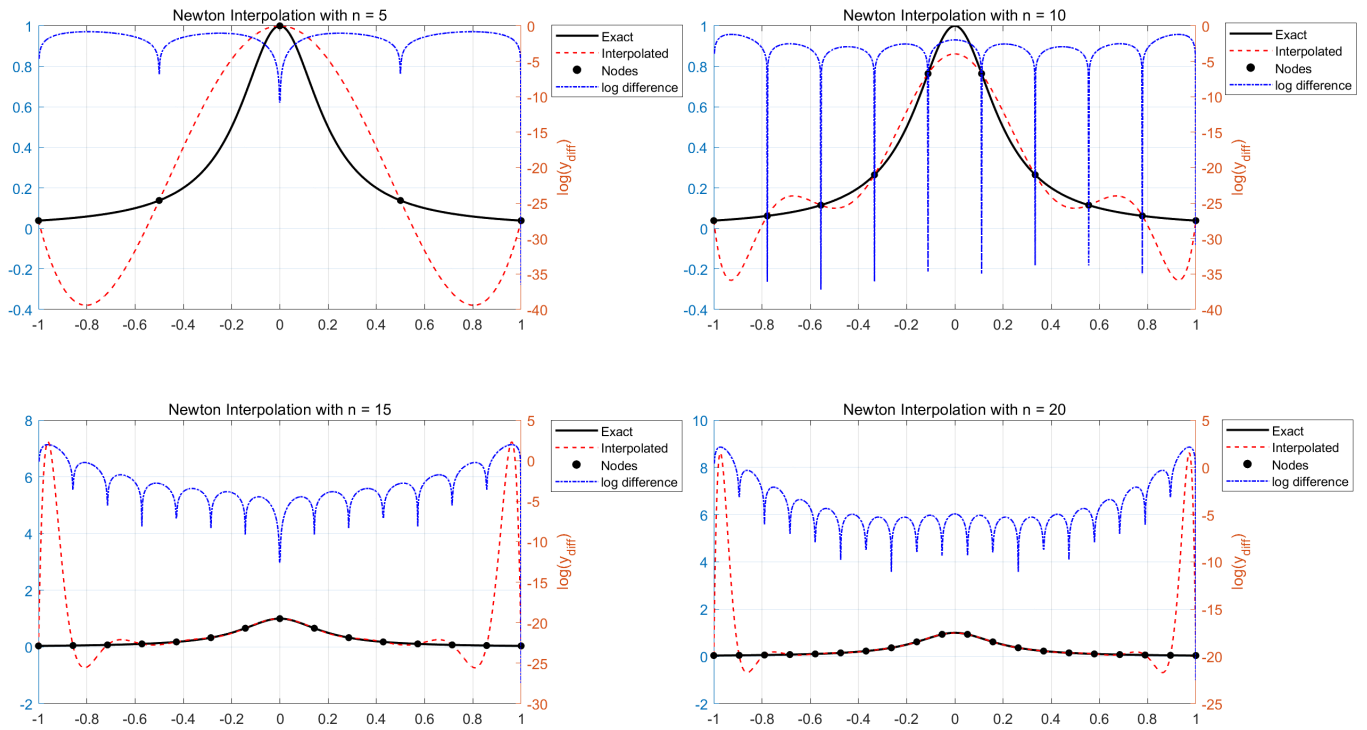
figure;
for i = 1:length(ns_g)
    subplot(2,2,i);
    newton_interpolation(g, ns_g(i), [-1, 1]);
end

```

运行结果:



出现了 Runge 现象 (即当节点增加时反而不能更好地接近被插函数的现象):



(2) 等距节点的近似 Newton 插值

由于是等距取样，利用差商和差分的关系，我们可以将 Newton 插值公式重写为以下近似形式：

- ① 表初公式: (用于计算最小节点 x_0 附近的插值)
 设 $x = x_0 + ph$ ，于是 $x - x_i = (p - i)h$

$$\begin{aligned}
 P(x) &= f(x_0) + f[x_0, x_1](x - x_0) + \cdots + f[x_0, x_1, \dots, x_k](x - x_0)(x - x_1) \cdots (x - x_{k-1}) \\
 &\quad (\text{note that } f[x_0, x_1, \dots, x_k] = \frac{\Delta^k f_0}{h^k k!}) \\
 &= f_0 + \frac{\Delta f_0}{h} \cdot ph + \frac{\Delta^2 f_0}{h^2 2!} \cdot ph \cdot (p-1)h + \cdots + \frac{\Delta^k f_0}{h^k k!} \cdot ph \cdot (p-1)h \cdots (p-k+1)h \\
 &= \binom{p}{0} f_0 + \binom{p}{1} \Delta f_0 + \binom{p}{2} \Delta^2 f_0 + \cdots + \binom{p}{k} \Delta^k f_0 \\
 &= \sum_{i=0}^k \binom{p}{i} \Delta^i f_0 \\
 &\quad \text{where } \binom{p}{i} := \frac{p(p-1) \cdots (p-i+1)}{i!}
 \end{aligned}$$

```

% 前向插值公式
function y_interp = newton_forward(p, delta_f)
    % 初始化插值结果为差分表的第一项
    y_interp = delta_f(1);
    % 初始化二项式系数
    binom = 1;

    % 遍历差分表，累加插值项
    for k = 1:length(delta_f)-1
        % 更新二项式系数
        binom = binom * (p-(k-1)) / k;
        % 累加插值项
        y_interp = y_interp + binom * delta_f(k+1);
    end

```

```
end
end
```

- ② 表末公式: (用于计算最大节点 x_n 附近的插值)

设 $x = x_n - ph$, 于是 $x - x_i = (p - i + n)h$

$$P(x) = f(x_n) + f[x_n, x_{n-1}](x - x_n) + \cdots + f[x_n, x_{n-1}, \dots, x_{n-k}](x - x_n)(x - x_{n-1}) \cdots (x - x_{n-k+1})$$

$$(\text{note that } f[x_n, x_{n-1}, \dots, x_{n-k}] = \frac{\Delta^k f_{n-k}}{h^k k!})$$

$$= f_n + \frac{\Delta f_{n-1}}{h}(-ph) + \frac{\Delta^2 f_{n-2}}{h^2 2!} \cdot (-ph) \cdot [-(p-1)h] + \cdots + \frac{\Delta^k f_{n-k}}{h^k k!}(-ph) \cdot [-(p-1)h] \cdots [-(p-k+1)h]$$

$$= \binom{p}{0} f_n - \binom{p}{1} \Delta f_{n-1} + \binom{p}{2} \Delta^2 f_{n-2} + \cdots + (-1)^k \binom{p}{k} \Delta^k f_{n-k}$$

$$= \sum_{i=0}^k (-1)^i \binom{p}{i} \Delta^i f_{n-i}$$

$$\text{where } \binom{p}{i} := \frac{p(p-1) \cdots (p-i+1)}{i!}$$

% 后向插值公式

```
function y_interp = newton_backward(p, delta_f)
```

```
% 初始化插值结果为差分表的最后一项
```

```
y_interp = delta_f(end, 1);
```

```
% 初始化二项式系数
```

```
binom = 1;
```

```
% 遍历差分表, 累加插值项
```

```
for k = 1:size(delta_f, 1) - 1
```

```
% 更新二项式系数
```

```
binom = -binom * (p-(k-1)) / k;
```

```
% 累加插值项
```

```
y_interp = y_interp + binom * delta_f(end-k, k+1);
```

```
end
```

```
end
```

- ③ Stirling 公式: (用于计算中间节点 x_m 附近的插值)

设 $x = x_m + ph$ ([Stirling Interpolation Formula](#))

$$P(x) = f_m + \frac{p}{2}(\Delta f_m + \Delta f_{m-1}) + \frac{p^2}{2} \Delta^2 f_{m-1} + \frac{(p-1)p(p+1)}{6} \left(\frac{\Delta^3 f_{m-1} + \Delta^3 f_{m-2}}{2} \right) + \cdots$$

% Stirling 中心插值公式

```
function y_interp = newton_stirling(p, delta_f)
```

```
% 获取差分表的大小
```

```
n = size(delta_f, 1);
```

```
% 计算中心节点索引
```

```
m = (n+1)/2;
```

```
% 使用 Stirling 公式计算插值
```

```
y_interp = delta_f(m, 1) + ( ...
    p/2 * (delta_f(m, 2) + delta_f(m-1, 2)) + ( ...
    p^2/2 * delta_f(m-1, 3) + ( ...
    (p-1)*p*(p+1)/12 * (delta_f(m-2, 4) + delta_f(m-1, 4)));
```

```
end
```

计算差分表的函数:

```
% 计算差分表的函数
```



```

function delta_f = differences(y)
    % 获取数据点的数量
    n = length(y);
    % 初始化差分表
    delta_f = zeros(n, n);
    % 第一列为原始数据
    delta_f(:, 1) = y(:);

    % 计算差分表
    for j = 2:n
        for i = 1:n-j+1
            % 递归计算差分
            delta_f(i, j) = delta_f(i+1, j-1) - delta_f(i, j-1);
        end
    end
end

```

计算近似 Newton 插值的函数:

```

function newton_interpolation(f, n, interval)
    x_exact = linspace(interval(1), interval(2), 1000);
    x_nodes = linspace(interval(1), interval(2), n);
    y_exact = f(x_exact); % 精确值
    y_nodes = f(x_nodes); % 等距采样
    y_interp = zeros(size(x_exact)); % 等距采样的近似 Newton 插值结果

    % 计算差分表
    delta_f = differences(y_nodes);
    % 计算节点间距 h
    h = x_nodes(2) - x_nodes(1);

    for j = 1:length(x_exact)
        % 如果 x_exact(j) 在 x_1 和 x_n 之间, 使用前向插值公式 (表初公式)
        if x_exact(j) <= x_nodes(min(3,n))
            % 假设 x = x_1 + p * h
            p = (x_exact(j) - x_nodes(1)) / h;
            % 至多到一个4阶差分, 因此只取5个样本
            y_interp(j) = newton_forward(p, delta_f(1, 1: min(5, n)));

            % 如果 x_exact(j) 在 x_{n-2} 和 x_n 之间, 使用后向插值公式 (表末公式)
            elseif x_exact(j) >= x_nodes(max(1,n-2))
                % 假设 x = x_n - p * h
                p = (x_nodes(n) - x_exact(j)) / h;
                % 至多到一个4阶差分, 因此只取5个样本
                y_interp(j) = newton_backward(p, delta_f(max(1, n-4):n, 1: n-max(1, n-4)+1));

            % 对于中间的位置, 使用 Stirling 中心插值公式
            else
                % 选取 x 左侧离它最近的节点 x_m
                m = floor((x_exact(j) - x_nodes(1)) / h) + 1;
                % 假设 x = x_m + p * h
                p = (x_exact(j) - x_nodes(m)) / h;
                % 至多到两个3阶差分, 因此只取5个样本
                y_interp(j) = newton_stirling(p, delta_f(m-2:m+2, 1:5));
            end
        end
    end

    % 插值与精确值的差距

```

```

y_diff = abs(y_interp - y_exact);

% 绘图
% 左侧纵轴：绘制精确值和插值结果
yyaxis left;
plot(x_exact, y_exact, 'k-', 'Linewidth', 1.5); hold on;
plot(x_exact, y_interp, 'r--', 'Linewidth', 1);
scatter(x_nodes, y_nodes, 25, 'k', 'filled');
title(['Newton Interpolation with n = ', num2str(n)]);
hold off;

% 右侧纵轴：绘制 log(y_diff)
yyaxis right;
plot(x_exact, log(y_diff), 'b-.', 'Linewidth', 1);
ylabel('log(y_{diff})');
legend('Exact', 'Interpolated', 'Nodes', 'log difference');
grid on;
end

```

函数调用:

```

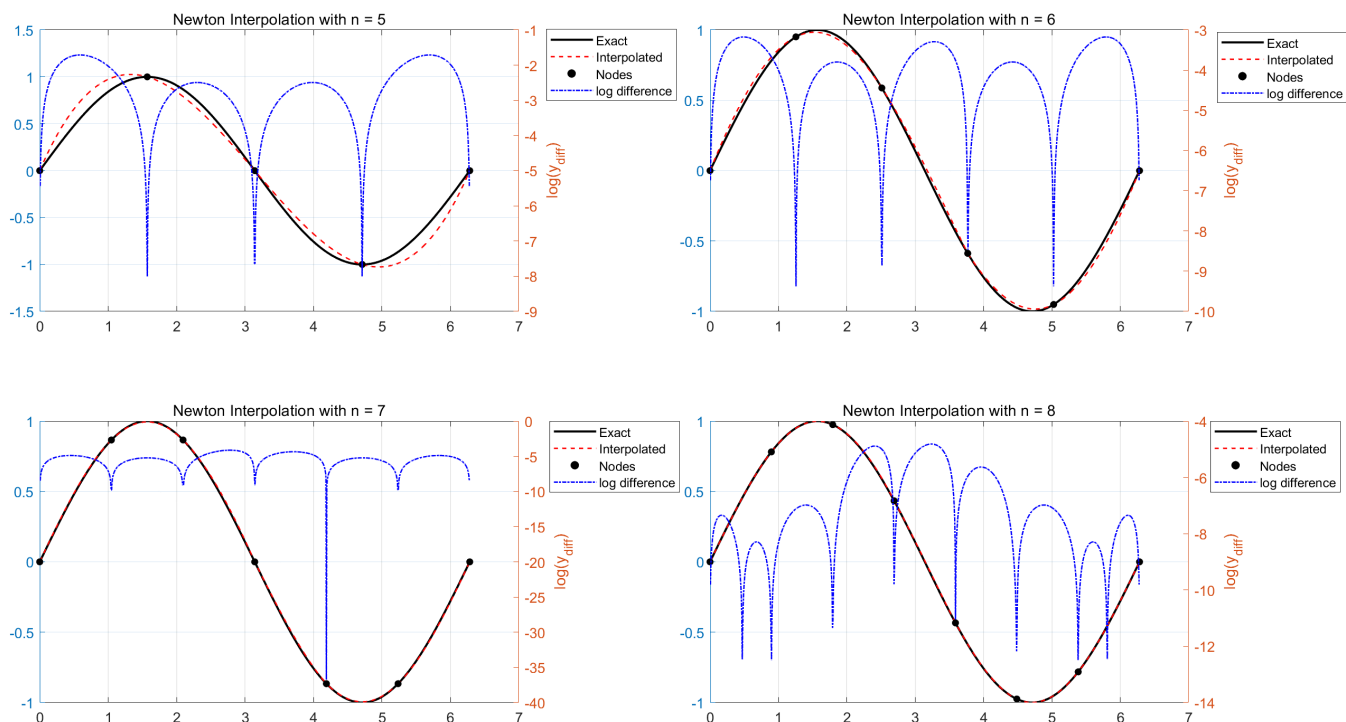
f = @(x) sin(x);
g = @(x) 1 ./ (1 + 25 * x.^2);
ns_f = [5, 6, 7, 8];
ns_g = [5, 10, 15, 20];

figure;
for i = 1:length(ns_f)
    subplot(2,2,i);
    newton_interpolation(f, ns_f(i), [0, 2*pi]);
end

figure;
for i = 1:length(ns_g)
    subplot(2,2,i);
    newton_interpolation(g, ns_g(i), [-1, 1]);
end

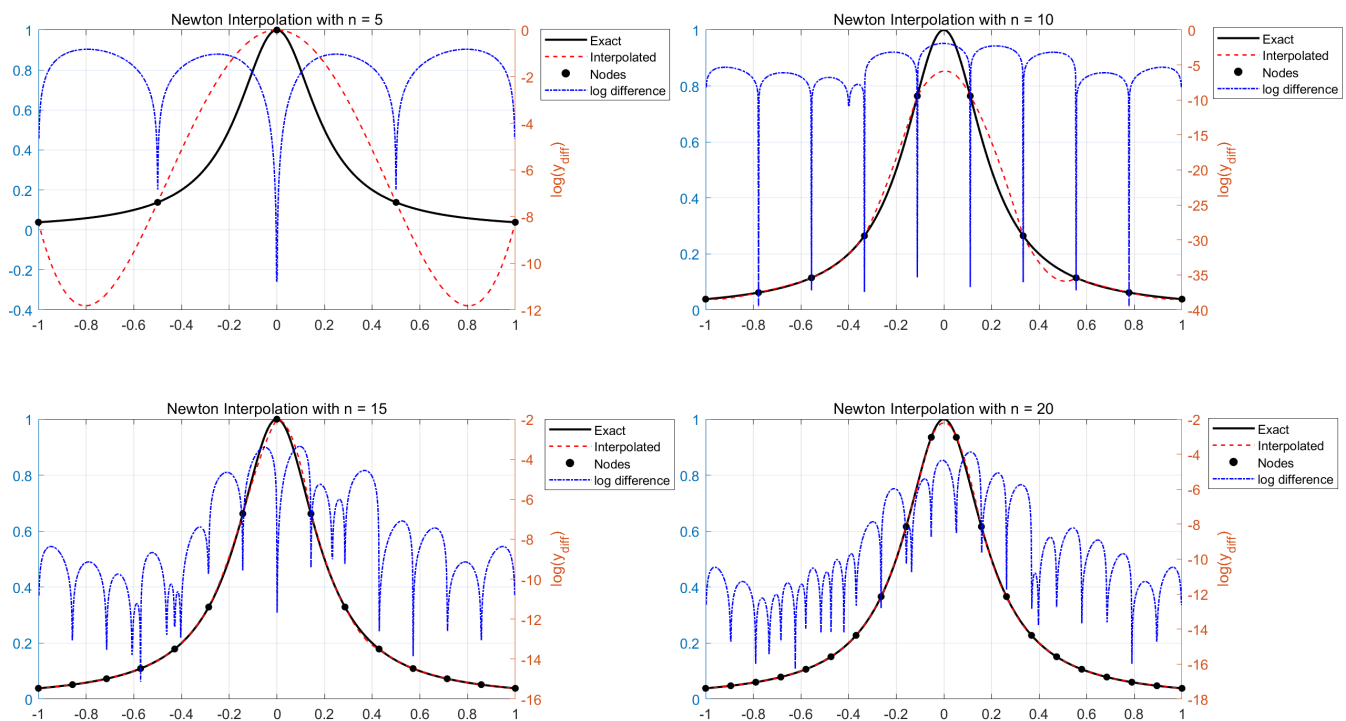
```

运行结果:



Runge 现象消失了:

(待改进: 插值结果不中心对称, 需要修改 Stirling 插值公式中 m 的选取逻辑)



Problem 5

(非典型的二维插值方法)

This exercise is about an atypical approach for two-dimensional interpolation.

Interpolating a data set $\{(x_i, y_i, z_i)\}_{i=1}^n \subset \mathbb{R}^3$ can be understood as interpolating $\{(x_i + iy_i), z_i\}_{i=1}^n \subset \mathbb{C} \times \mathbb{R}$, where the interpolation nodes $x_i + iy_i$'s are complex numbers.

The polynomial interpolation techniques we have learned theoretically carry over to complex inputs,

while the resulting interpolation polynomial is in general complex-valued.

Nevertheless, we can take the real part of the output.

Use this approach to interpolate the following data set over the unit disk and visualize the result.

If you use MATLAB/Octave, the functions `imagesc` and `colorbar` can be useful for visualizing a bivariate function.

x_i	y_i	z_i
1.00000	0.00000	-1.0000
0.80902	0.58779	-2.6807
0.30902	0.95106	5.6161
-0.30902	0.95106	5.6161
-0.80902	0.58779	-2.6807
-1.00000	0.00000	-1.0000
-0.80902	-0.58779	-2.6807
-0.30902	-0.95106	5.6161
0.30902	-0.95106	5.6161
0.80902	-0.58779	-2.6807

- 我们一般不把二元多项式插值问题转化为复平面上的多项式插值问题。
因为复多项式是解析函数，其实部是调和函数，具有很多好的性质，例如极值原理（调和函数的极值在边界取得）但一般的函数不一定具有这样好的性质（如存在局部极值点或非光滑区域），因此插值结果往往是面目全非的。
- 我们一般使用其他方法处理二元插值问题，例如 Shepard 插值。

Solution:

Newton 插值公式:

$$\begin{aligned} P(x) &= f(x_1) + \sum_{k=2}^n f[x_1, x_2, \dots, x_k] \prod_{i=1}^{k-1} (x - x_i) \\ &= f(x_1) + f[x_1, x_2](x - x_1) + \dots + f[x_1, x_2, \dots, x_n](x - x_1)(x - x_2) \cdots (x - x_{n-1}) \\ \text{where } f[x_1, x_2, \dots, x_k] &:= \sum_{i=1}^k y_i \prod_{j \neq i} \frac{1}{x_i - x_j} \end{aligned}$$

实际计算时，差商不是直接通过 $f[x_1, x_2, \dots, x_k] := \sum_{i=1}^k y_i \prod_{j \neq i} \frac{1}{x_i - x_j}$ 计算的（否则就是 Lagrange 插值了）它是从 $f[x_k] = f(x_k) = y_k$ 出发递归地求解 $f[x_1, \dots, x_k]$:

$$f[x_1, \dots, x_k] = \frac{f[x_2, \dots, x_k] - f[x_1, \dots, x_{k-1}]}{x_k - x_1}$$

本题只需将自变量 x 视为复数，因变量取插值结果的实部即可。

计算差商表的函数:

```
% 计算差商表的函数
function delta_f = divided_differences(x, y)
    % 获取数据点的数量
    n = length(y);
    % 初始化差商表
    delta_f = zeros(n, n);
    % 第一列为原始数据
    delta_f(:, 1) = y(:);

    % 计算差商表
    for j = 2:n
        for i = 1:n-j+1
```

```

        % 递归计算差商
        delta_f(i, j) = (delta_f(i+1, j-1) - delta_f(i, j-1)) / (x(j+i-1)-x(i));
    end
end

% 返回差商表的第一行
delta_f = delta_f(1,:);
end

```

函数调用:

```

nodes = [1.00000,  0.00000, -1.0000;
         0.80902,  0.58779, -2.6807;
         0.30902,  0.95106,  5.6161;
        -0.30902,  0.95106,  5.6161;
        -0.80902,  0.58779, -2.6807;
        -1.00000,  0.00000, -1.0000;
        -0.80902, -0.58779, -2.6807;
        -0.30902, -0.95106,  5.6161;
         0.30902, -0.95106,  5.6161;
         0.80902, -0.58779, -2.6807];

% 输入与输出
x_nodes = nodes(:,1) + 1i*nodes(:,2);
y_nodes = nodes(:,3);

% 计算差商表 (的第一行)
delta_f = divided_differences(x_nodes, y_nodes);

% 定义复平面上的网格点
[x_real, x_imag] = meshgrid(linspace(-1, 1, 1000), linspace(-1, 1, 1000));
x = x_real(:) + 1i*x_imag(:); % 将网格点转化为复数

% 初始化插值结果
y_interp = zeros(size(x)); % 插值结果初始化

% 迭代项
term = ones(size(x)); % 初始化累乘项

% Newton 插值公式
for j = 1:length(y_nodes)
    y_interp = y_interp + delta_f(j) * term; % 累加插值项
    term = term .* (x - x_nodes(j)); % 更新累乘项
end

% 将插值结果重塑为网格形式
y_interp = reshape(y_interp, size(x_real));

% 提取插值结果的实部 (如果需要)
y_interp_real = real(y_interp);

% 绘制三维图像
figure;
surf(x_real, x_imag, y_interp_real, 'EdgeColor', 'none'); % 绘制插值曲面
hold on;
scatter3(real(x_nodes), imag(x_nodes), y_nodes, 25, 'k', 'filled'); % 绘制原始数据点
title('Newton Interpolation on Complex Plane');
xlabel('Real Part');

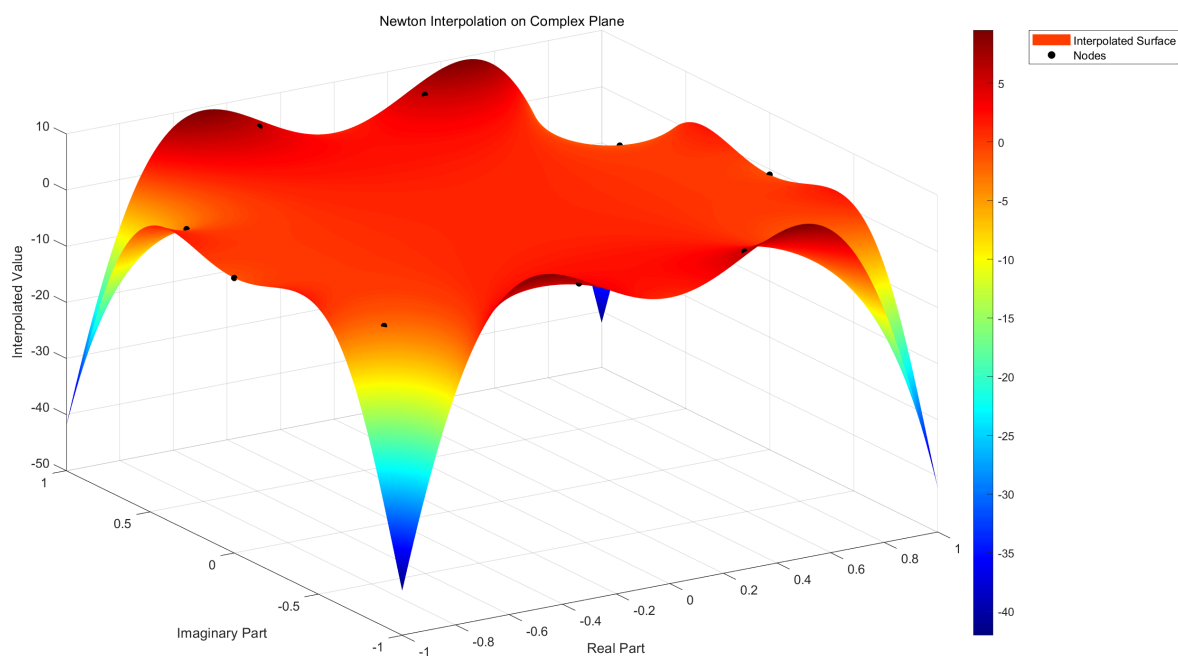
```

```

ylabel('Imaginary Part');
xlabel('Interpolated Value');
colormap jet; % 设置颜色映射
colorbar; % 添加颜色条
legend('Interpolated Surface', 'Nodes');
grid on;
hold off;

```

运行结果:



Problem 6 (optional)

Let $a_0, a_1, \dots, a_{n-1}, a_0, a_1, \dots, a_{n-1}, \dots, a_0, a_1, \dots, a_{n-1}, \dots$ be a periodic sequence. Can you find a smooth function that interpolates this infinite sequence?

Solution:

受 Lagrange 插值公式的启发, 我们定义函数:

$$g_k(x) := \prod_{j \neq k} \frac{\exp(i \frac{2\pi x}{n}) - \exp(i \frac{2\pi j}{n})}{\exp(i \frac{2\pi k}{n}) - \exp(i \frac{2\pi j}{n})} \quad (k = 0, 1, \dots, n-1)$$

对于 $j = 0, 1, \dots, n-1$, 这一系列函数满足:

$$q_k(j) = \begin{cases} 1, & (j \bmod n) = k \\ 0, & (j \bmod n) \neq k \end{cases} \quad (k = 0, 1, \dots, n-1)$$

据此我们可以定义这样一个周期为 n 的光滑函数:

$$g(x) := \sum_{k=0}^{n-1} a_k g_k(x) \quad (\forall x \in \mathbb{R})$$

它满足:

$$g(k + pn) = a_k \quad (\forall k = 0, 1, \dots, n-1 \text{ and } p \in \mathbb{Z})$$

Problem 7 (optional)

Polynomial interpolation are useful not only in modeling, but also in pure mathematics. For instance, the Chinese Remainder Theorem can be derived from Lagrange's approach. This exercise is another example.

Let $f(x)$ be a real polynomial of degree n .

Suppose that there exists $i \in \mathbb{Z}$ such that $f(i), f(i+1), \dots, f(i+n)$ are all integers.

- (a) Show that $f(x) \in \mathbb{Z}$ for any $x \in \mathbb{Z}$.
- (b) Is it also true that $f(x) \in \mathbb{Z}[x]$, i.e., $f(x)$ has integer coefficients?

Part (a)

Let $f(x)$ be a real polynomial of degree n .

Suppose that there exists $i \in \mathbb{Z}$ such that $f(i), f(i+1), \dots, f(i+n)$ are all integers.

Show that $f(x) \in \mathbb{Z}$ for any $x \in \mathbb{Z}$.

Solution:

不失一般性, 设 $i = 0$, $f(0), f(1), \dots, f(n)$ 均为整数.

我们定义:

$$l_k(x) := \prod_{\substack{0 \leq j \leq n \\ j \neq k}} \frac{x-j}{k-j}$$

由于 $f(x)$ 是 n 次实系数多项式, 故 n 次插值多项式即为 $f(x)$:

$$f(x) = \sum_{k=0}^n f(k)l_k(x)$$

注意到:

- ① 当 x 为整数且 $0 \leq x \leq n$ 时, 我们有:

$$l_k(x) = \begin{cases} 1, & \text{if } x = k \\ 0, & \text{if } x \in \{0, 1, \dots, n\} \setminus \{k\} \end{cases}$$

- ② 当 x 为整数且 $x \geq n+1$ 时, 我们有:

$$\begin{aligned} l_k(x) &= \prod_{\substack{0 \leq j \leq n \\ j \neq k}} \frac{x-j}{k-j} \\ &= \frac{x \cdot (x-1) \cdots (x-k+1) \cdot (x-k-1) \cdots (x-n)}{k \cdot (k-1) \cdots 1 \cdot (-1) \cdot (-2) \cdots (k-n)} \\ &= \frac{x! / [(x-k) \cdot (x-n-1)!]}{k! \cdot (-1)^{n-k} \cdot (n-k)!} \\ &= (-1)^{n-k} \frac{x!}{k! \cdot (n-k)! \cdot (x-n-1)! \cdot (x-k)} \\ &= (-1)^{n-k} \cdot \frac{x!}{k!(x-k)!} \cdot \frac{(x-k-1)!}{(n-k)!(x-n-1)!} \\ &= (-1)^{n-k} \cdot \binom{x}{k} \cdot \binom{x-k-1}{n-k} \in \mathbb{Z} \end{aligned}$$

- ③ 当 x 为整数且 $x \leq -1$ 时, 我们有:

$$\begin{aligned}
l_k(x) &= \prod_{\substack{0 \leq j \leq n \\ j \neq k}} \frac{x-j}{k-j} \\
&= \frac{x \cdot (x-1) \cdots (x-k+1) \cdot (x-k-1) \cdots (x-n)}{k \cdot (k-1) \cdots 1 \cdot (-1) \cdot (-2) \cdots (k-n)} \\
&= \frac{(-1)^{n+1} \cdot (n-x)! / [(k-x) \cdot (-x-1)!]}{k! \cdot (-1)^{n-k} \cdot (n-k)!} \\
&= (-1)^{2n-k+1} \frac{(n-x)!}{k! \cdot (n-k)! \cdot (-x-1)! \cdot (k-x)} \\
&= (-1)^{1-k} \cdot \frac{(n-x)!}{(n-k)! (k-x)!} \cdot \frac{(k-1-x)!}{k! (-x-1)!} \\
&= (-1)^{1-k} \cdot \binom{n-x}{n-k} \cdot \binom{k-1-x}{k} \in \mathbb{Z}
\end{aligned}$$

综合 ①②③ 可知, 对于任意 $x \in \mathbb{Z}$ 我们都有 $l_k(x) \in \mathbb{Z}$

因此对于任意 $x \in \mathbb{Z}$ 我们都有 $f(x) = \sum_{k=0}^n f(k)l_k(x) \in \mathbb{Z}$ 成立.

Part (b)

Let $f(x)$ be a real polynomial of degree n .

Suppose that there exists $i \in \mathbb{Z}$ such that $f(i), f(i+1), \dots, f(i+n)$ are all integers.

Is it also true that $f(x) \in \mathbb{Z}[x]$, i.e., $f(x)$ has integer coefficients?

Solution:

可以举例说明 $f(x)$ 不一定是整系数 n 次多项式:

取 $n = 2$, 设 $f(0) = 1, f(1) = 0, f(2) = 0$, 我们有:

$$\begin{aligned}
f(x) &= f(0) \cdot \frac{(x-1)(x-2)}{(0-1)(0-2)} + f(1) \cdot \frac{(x-0)(x-2)}{(1-0)(1-2)} + f(2) \cdot \frac{(x-0)(x-1)}{(2-0)(2-1)} \\
&= 1 \cdot \frac{(x-1)(x-2)}{2} + 0 \cdot \frac{x(x-2)}{-1} + 0 \cdot \frac{x(x-1)}{2} \\
&= \frac{1}{2}x^2 - \frac{3}{2}x + 1
\end{aligned}$$

而 $f(x)$ 并不是整系数 2 次多项式.

The End